



PRO API MANUAL

Flask Pro API, auth, /run contract, preview, backtest

Open-source Pine Script evaluation framework



CONTENTS

01	Pro API
02	App Lifecycle
03	Auth and Keys
04	POST /run
05	Preview Endpoints
06	Backtest Endpoint
07	Chart Renderer
08	Runtime Bridge
09	Evaluate Contract



PRO API

Flask Pro API for Pine evaluation — /run, preview, backtest, API keys, and the shared evaluate contract.

PRO API

The PYNE **Pro API** is a Flask HTTP service that runs Pine scripts over caller-supplied OHLCV, returns plots / series / strategy events / drawings, and gates thumbnail + backtest routes behind API keys. It is intentionally **not** the language server: different auth, scaling, and failure domains.

Abstract

Where the LSP answers *static* editor questions, the Pro API answers *dynamic* evaluate questions:

```
POST JSON { script, data: OHLCV[], ... }  
  → backend.runtime.Runtime.run  
  → { plots, series, plot_meta, events, drawings, script_id, run_id, ... }
```

Free-tier evaluate (`POST /run` , `POST /run/batch`) validates bodies with hand-rolled schemas, enforces a 5 MB body cap, and restricts CORS. Pro routes under `/preview/*` and `/backtest/*` use `track_usage` (Bearer / ApiKey / query key). Chart PNGs are matplotlib Agg renders encoded as base64.

The same **evaluate contract** is mirrored by edge workers (pyne-worker / pine-worker) so AXIS and HOOX can swap hosts without redesigning payloads — see [contract](#).

Conceptual model

```
flowchart TB  
  CLIENT[AXIS / CLI / curl] --> FLASK[backend.app Flask]  
  FLASK --> AUTH[middleware auth + schemas]  
  FLASK --> RUN["/run · /run/batch"]  
  FLASK --> PREV["/preview/*"]  
  FLASK --> BT["/backtest/*"]  
  RUN --> RT[backend.runtime.Runtime]  
  PREV --> CHART[chart_renderer]  
  BT --> BACK[services.backtest]  
  RT --> EVAL[CustomEvaluator + PineSeries]  
  RT --> AST[parse ASDL]
```



Interface surface

Method	Path	Auth	Role
GET	/	none	Health + endpoint map
POST	/run	none (free)	Single-script evaluate
POST	/run/batch	none (free)	≤8 scripts, shared OHLCV
POST	/preview/chart	API key + usage	Line / OHLCV PNG
POST	/preview/indicator	API key + usage	Expression series PNG
POST	/backtest/quick	API key + usage	Quick strategy metrics + equity PNG
POST	/auth/create_key	admin decorator	Mint <code>pyn_...</code> key
GET	/auth/usage	API key	Usage counters
POST	/auth/validate	none (body key)	Validate without consuming quota

Local runner: `make run` → `python -m backend.app` (default `127.0.0.1:5002`).

Tracks in this tab

1. [App lifecycle](#) — Flask app, CORS, limits, blueprints
2. [Auth and keys](#) — tiers, stores, decorators
3. [Run](#) · [Preview](#) · [Backtest](#)
4. [Chart renderer](#)
5. [Runtime bridge](#) — `Runtime` / series / evaluator glue
6. [Contract](#) — shared evaluate schema across Flask and workers

Internals (map)

Path	Role
<code>backend/app.py</code>	App factory surface, <code>/run*</code> , <code>/auth*</code> , CORS
<code>backend/api/preview.py</code>	Preview + backtest blueprints
<code>backend/middleware/*</code>	Auth, schemas, optional Redis/SQLite stores
<code>backend/runtime.py</code>	Bar-loop evaluate façade
<code>backend/evaluator.py</code>	<code>CustomEvaluator</code> bridge
<code>backend/series.py</code>	<code>PineSeries</code> history deque
<code>backend/services/*</code>	Charts + backtest simulation

See also

- [Evaluate scripts \(end user\)](#)
- [Pro API usage guide](#)
- [Runtime hub](#)
- [LSP hub](#) — editor path, not HTTP



APP LIFECYCLE

Flask application setup: MAX_CONTENT_LENGTH, CORS policy, blueprints, error handlers, and process entry.

APP LIFECYCLE

Abstract

`backend/app.py` constructs a single global `Flask` app, applies security-minded defaults (body size, CORS allowlist), registers free and Pro routes, and exposes a module `__main__` for local development. There is no multi-app factory pattern today — import `app` for WSGI (gunicorn) or run the module for the built-in server.

Conceptual model

```
flowchart TB
    BOOT[import backend.app] --> CFG[MAX_CONTENT_LENGTH + CORS]
    CFG --> ROUTES[register / · /run · /auth]
    ROUTES --> BP[register preview_bp + backtest_bp]
    BP --> ERR[404 / 500 JSON handlers]
    ERR --> SERVE{entry}
    SERVE -->|gunicorn| WSGI[app:app]
    SERVE -->|python -m backend.app| DEV[HOST/PORT env]
```

Interface surface

Process entry

```
make run
# equivalent
python -m backend.app
```

Environment:

Variable	Default	Role
HOST	127.0.0.1	Bind address (localhost-first for dev)
PORT	5002	Listen port
ALLOWED_ORIGINS	https://pynescrypt.ai , https://app.pynescrypt.ai , localhost regex	CORS origins (comma-separated; regex allowed)
API_KEY_STORE	/root/pynescrypt/data/api_keys.json	JSON key store path (file-backed default)
ADMIN_TOKEN	unset	Documented for admin create_key (see auth)

`debug=False` always on the dev runner.



Health

GET / → JSON:

```
{
  "status": "healthy",
  "service": "pynescrypt-pro-api",
  "version": "1.0.0",
  "timestamp": 0,
  "endpoints": { "...": "..." }
}
```

Hard limits

- `MAX_CONTENT_LENGTH = 5 * 1024 * 1024` — reject oversized bodies before JSON parse fills memory (audit 2026-07-05 / S1).
- CORS methods: `GET`, `POST` only.
- Allow headers: `Content-Type`, `Authorization`, `X-Admin-Token`.
- `supports_credentials=False`.

Localhost regex used in the default origin list:

```
^https?:/(?:localhost|127\.0\.0\.1)(?:\d+)?$
```

Same-origin / no `Origin` header (curl, server-to-server) remains usable under flask-cors behavior.

Blueprints

```
app.register_blueprint(preview_bp)    # url_prefix=/preview
app.register_blueprint(backtest_bp)   # url_prefix=/backtest
```

Defined in `backend/api/preview.py`.

Error handlers

Code	Body code	Message
404	<code>NOT_FOUND</code>	Endpoint {path} not found.
500	<code>INTERNAL_ERROR</code>	Internal server error.

Both return JSON with `status: "error"`.

Internals

Path	Role
<code>backend/app.py</code>	App object, free routes, auth routes
<code>backend/api/preview.py</code>	Pro blueprints
<code>backend/middleware/schemas.py</code>	Request validation for <code>/run*</code> and auth
<code>backend/requirements.txt</code>	Flask, flask-cors, numpy, matplotlib, ...



Recommended production: gunicorn/uvicorn-class WSGI with multiple workers; for multi-worker key consistency prefer SQLite/Redis stores over the default in-memory/file singleton assumptions — see [auth](#).

Invariants and edge cases

- 1. **Import side effects:** creating `app` configures CORS immediately from env.
- 2. **Body too large** → Flask 413 before route logic (not custom JSON).
- 3. **Unknown fields** on schema-validated routes → `UNKNOWN_FIELDS 400 ( /run , auth)`; preview routes currently use looser `get_json()` (schemas exist but are not always applied in handlers).
- 4. **Dev bind default is loopback** — set `HOST=0.0.0.0` only when intentional.

Worked example — smoke test

```
curl -s http://127.0.0.1:5002/ | jq .status
# "healthy"
```

Failure modes

Symptom	Cause
Browser CORS errors	Origin not in <code>ALLOWED_ORIGINS</code>
413 on large OHLCV	Exceeded 5 MB — downsample or paginate
Import errors for matplotlib	Missing backend deps — <code>pip install -r backend/requirements.txt</code>

See also

- [Auth and keys](#)
- [Run endpoint](#)
- [Docker / CI](#)



AUTH AND KEYS

API key model, tiers, require_api_key / track_usage decorators, JSON/SQLite/Redis stores, and admin create_key.

AUTH AND KEYS

Abstract

Pro routes authenticate with a **raw API key** presented as `Authorization: Bearer ...`, `Authorization: ApiKey ...`, or `?api_key=`. Keys are minted as `pyn_` + url-safe secret, stored with a short `key_id` and SHA-256 hash metadata, and tracked against **monthly-ish call limits** by tier. Free `/run` does not require a key.

Conceptual model

```
flowchart LR
    REQ[HTTP request] --> HDR{Authorization / query}
    HDR --> VAL[store.validate_key]
    VAL -->|miss| U401[401 UNAUTHORIZED]
    VAL -->|hit + over limit| U429[429 RATE_LIMITED]
    VAL -->|ok| G[g.api_key]
    G --> H[handler]
    H --> TRACK[increment_calls on success]
```

Interface surface

Tiers (`_TIER_LIMITS`)

Tier	calls_limit
free	0 (treated as unlimited in <code>is_rate_limited</code> — limit 0 means “no cap”)
hobby	5 000
pro	50 000
team	200 000
enterprise	inf

`calls_remaining()` returns `inf` when limit is 0.

Endpoints

POST `/auth/create_key`

Body schema `CREATE_KEY_SCHEMA`: optional `tier` (default `"hobby"`).

Response:



```
{
  "status": "success",
  "api_key": "pyn_...",
  "key_id": "12hexchars",
  "tier": "hobby",
  "message": "Store this API key securely. It will not be shown again."
}
```

Decorated with `@require_admin_token`. **Implementation note:** `require_admin_token` currently sets `g.is_admin = True` unconditionally (placeholder); production deployments should enforce `ADMIN_TOKEN` / role checks before exposing this route. App docstring describes `X-Admin-Token` + `ADMIN_TOKEN` env as the intended design.

GET /auth/usage

`@require_api_key` — returns `key_id`, `tier`, and usage block (`calls_used`, `calls_limit`, `calls_remaining`, timestamps).

POST /auth/validate

Body: `{ "api_key": "..." }` — validates without incrementing usage. Returns tier info, `active`, `rate_limited`.

Decorators

Decorator	Behavior
<code>require_api_key</code>	Resolve key → 401 / 429 or set <code>g.api_key</code>
<code>track_usage</code>	Wraps <code>require_api_key</code> ; increments calls when response status < 400 (tuple responses) or always for bare Response
<code>require_admin_token</code>	Placeholder admin gate for <code>create_key</code>

Client presentation

```
curl -H "Authorization: Bearer $API_KEY" \
  -H "Content-Type: application/json" \
  -d @body.json \
  http://127.0.0.1:5002/preview/chart
```

Internals

APIKey dataclass

Fields: `key_id`, `key_hash`, `tier`, `calls_used`, `calls_limit`, `created_at`, `last_used`.

Helpers: `is_active()` (always True today), `is_rate_limited()`, `get_tier_info()`, `increment_calls()`.

Stores

Module	Use
<code>middleware/auth.py</code> → <code>APIKeyStore</code>	Default JSON file store (<code>API_KEY_STORE</code> path)
<code>middleware/key_store_sqlite.py</code>	Hash-only rows, WAL, multi-worker friendly
<code>middleware/key_store_redis.py</code>	Redis-backed alternative



Default `get_key_store()` returns the JSON-file `APIKeyStore` singleton. JSON store currently keeps raw keys as dict keys in the file (dev convenience); SQLite module documents **hash-only** storage for safer leaks.

Key minting:

```
raw_key = "pyn_" + secrets.token_urlsafe(32)
key_id = sha256(raw_key).hexdigest()[:12]
key_hash = sha256(raw_key).hexdigest()
```

Schemas

`CREATE_KEY_SCHEMA`, `VALIDATE_KEY_SCHEMA` in `middleware/schemas.py` — strict types, reject unknown fields.

Invariants and edge cases

1. `/run` is **unauthenticated** — rate-limit at reverse proxy if exposed publicly.
2. **Limit 0 means unlimited**, not “zero calls” — free tier limit value is 0 with that semantics.
3. **Usage increments only after handler returns** — failed auth never bills; handler 4xx under `track_usage` skips increment when returned as `(response, status)`.
4. **Revocation** exists on the store API (`revoke_key`); no public HTTP revoke route in `app.py`.
5. **Admin decorator gap** — do not assume `create_key` is locked in all deployments until the TODO is closed.

Worked example

```
# create (when admin gate is open / local tests)
curl -s -X POST http://127.0.0.1:5002/auth/create_key \
  -H 'Content-Type: application/json' \
  -d '{"tier": "hobby"}'

# validate
curl -s -X POST http://127.0.0.1:5002/auth/validate \
  -H 'Content-Type: application/json' \
  -d '{"api_key": "pyn_..."}'
```

Failure modes

Code	Meaning
<code>UNAUTHORIZED</code>	Missing/invalid key
<code>RATE_LIMITED</code>	<code>calls_used >= calls_limit</code> (finite limits)
<code>INVALID_TIER</code>	<code>create_key</code> with unknown tier string
<code>INVALID_KEY</code>	validate endpoint miss

See also

- [App lifecycle](#)
- [Preview](#)
- [Security \(DevOps\)](#)



POST /RUN

Free evaluate endpoints: single-script /run and multi-script /run/batch over shared OHLCV.

POST `/run` AND `/run/batch`

Abstract

`/run` is the primary free evaluate entry: accept Pine source + bar list, optionally wire `request.*` data sources, execute via `Runtime`, and return plots/series/events/drawings. `/run/batch` runs up to eight scripts on the **same** OHLCV (AXIS multi-indicator), isolating per-script errors so one failure does not discard siblings.

Conceptual model

```
sequenceDiagram
    participant C as Client
    participant A as Flask /run
    participant V as validate RUN_SCHEMA
    participant D as resolve_request_sources
    participant R as Runtime.run

    C->>A: POST JSON
    A->>V: schema + required fields
    V->>D: optional data_source
    D->>R: script, ohlcv, mode
    R-->>C: success payload or EXECUTION_ERROR
```

Interface surface

POST `/run`

Request (`RUN_SCHEMA`)

Field	Type	Required	Default	Notes
<code>script</code>	string	yes		Pine source
<code>data</code>	list	yes		OHLCV bar objects
<code>symbol</code>	string	no	<code>"CHART"</code>	Syminfo / request wiring
<code>data_source</code>	string	no	<code>" "</code>	<code>mock</code> / <code>ccxt</code> / <code>yahoo</code> / ...
<code>data_options</code>	object	no	<code>{}</code>	Provider options
<code>mode</code>	string	no	<code>"interpret"</code>	<code>"interpret"</code> <code>"compile"</code>

Unknown fields → `UNKNOWN_FIELDS` 400. Empty script/data still checked after schema with `NO_SCRIPT` / `NO_DATA`.

Bar shape (runtime expectation):

```
{ "time": 0, "open": 1, "high": 1, "low": 1, "close": 1, "volume": 1 }
```



Optional `bid` / `ask` per bar update runtime bid/ask state.

Success response

```
{
  "status": "success",
  "plots": [],
  "series": {},
  "plot_meta": {},
  "events": [],
  "drawings": [],
  "script_id": "16hex",
  "run_id": "16hex",
  "count": 0,
  "mode": "interpret",
  "data_source": "chart"
}
```

Field	Meaning
plots	Primary series (first plot) — backward compatible list
series	Named multi-plot map title → values per bar
plot_meta	Per-title color, linewidth, index
events	Strategy events with <code>script_id</code> / <code>run_id</code> stamps
drawings	Exported line/label/box registry for AXIS
script_id	<code>sha256(source)[:16]</code>
run_id	Per- <code>Runtime</code> instance id

Errors

HTTP	code	When
400	<code>MISSING_FIELD</code> / <code>INVALID_FIELD</code> / <code>UNKNOWN_FIELDS</code> / <code>INVALID_BODY</code>	Schema
400	<code>NO_SCRIPT</code> / <code>NO_DATA</code>	Empty after defaults
400	<code>DATA_SOURCE_ERROR</code>	<code>resolve_request_sources</code> failed
500	<code>EXECUTION_ERROR</code>	Runtime returned <code>error</code> key

POST `/run/batch`

Request (`RUN_BATCH_SCHEMA`)

Field	Type	Required	Notes
scripts	list	yes	strings or <code>{id, script}</code> objects
data	list	yes	Shared OHLCV
symbol / <code>data_source</code> / <code>data_options</code> / <code>mode</code>	optional	same as <code>/run</code>	

Hard cap: `RUN_BATCH_MAX_SCRIPTS = 8` → `TOO_MANY_SCRIPTS` .

Response



```
{
  "status": "success" | "partial",
  "results": [ { "id": "...", "status": "success|error", ... } ],
  "count": 2,
  "ok": 1,
  "data_source": "chart"
}
```

HTTP **200** for partial success (per-script errors inside `results`). Envelope validation failures still 400.

Internals

```
runtime = Runtime(symbol=str(symbol))
result = runtime.run(script, ohlcv, data_feed=..., data_provider=..., mode=...)
```

See [runtime bridge](#) for bar-loop details and compile mode.

Paths: `backend/app.py` (`run_pine_script`, `run_pine_script_batch`), `backend/middleware/schemas.py`.

Invariants and edge cases

1. **No API key** on these routes — protect at the edge if needed.
2. **Batch isolates exceptions** per script (`try/except` around `runtime.run`).
3. **Whitespace-only `data_source`** coerced to `None` → chart default.
4. **Compile mode** requires numba; errors surface as `EXECUTION_ERROR` message from runtime.
5. Schema rejects extras — clients must not send AXIS-only fields without updating schema.

Worked example

```
curl -s http://127.0.0.1:5002/run \
-H 'Content-Type: application/json' \
-d '{
  "script": "//@version=5\nindicator(\"t\")\nplot(close)",
  "data": [
    {"time": 1, "open": 1, "high": 2, "low": 0.5, "close": 1.5, "volume": 10},
    {"time": 2, "open": 1.5, "high": 2.5, "low": 1.0, "close": 2.0, "volume": 12}
  ],
  "symbol": "TEST:DEMO"
}'
```

Failure modes

Symptom	Cause
<code>UNKNOWN_FIELDS</code>	Typo'd property (e.g. <code>ohlcv</code> instead of <code>data</code>)
Parse errors in message	Invalid Pine — same engine as CLI
Batch <code>status: partial</code>	At least one script failed; inspect <code>results[i].message</code>



See also

- [Contract](#)
- [Runtime bridge](#)
- [App lifecycle](#)



PREVIEW ENDPOINTS

Pro chart and indicator thumbnail routes under /preview with usage tracking.

PREVIEW ENDPOINTS

Abstract

Preview routes render **PNG thumbnails** (base64) for desk UIs and docs. They require an API key via `@track_usage`, accept OHLCV-shaped **objects** (column arrays), and do **not** run the full `Runtime` bar loop for `/preview/chart` (data is plotted directly). `/preview/indicator` evaluates a small set of closed-form `ta.*` expressions in pure Python for the series, then plots.

Conceptual model

```
flowchart LR
    KEY[API key] --> TRACK[track_usage]
    TRACK --> CHART["POST /preview/chart"]
    TRACK --> IND["POST /preview/indicator"]
    CHART --> R1[render_line_chart / render_ohlcw_chart]
    IND --> COMP[_compute_indicator]
    COMP --> R2[render_line_chart]
    R1 --> B64[base64 PNG]
    R2 --> B64
```

Blueprint: `preview_bp` with `url_prefix="/preview"` in `backend/api/preview.py`.

Interface surface

Auth

Both routes: `@track_usage` → must pass `require_api_key` and consume one call on success.

POST /preview/chart

Body (documented; schemas exist as `PREVIEW_CHART_SCHEMA`)

```
{
  "script": "optional string",
  "data": {
    "open": [], "high": [], "low": [], "close": [], "volume": []
  },
  "options": {
    "type": "line",
    "color": "#2196F3",
    "show_volume": false,
    "width": 600,
    "height": 300
  }
}
```



Option	Default	Clamp
type	"line"	"ohlcv" selects candlestick renderer
color	#2196F3	line chart only
width	600	max 1200
height	300	max 600
show_volume	false	twin-axis volume on line chart

Success

```
{
  "status": "success",
  "chart": "<base64 png>",
  "meta": {
    "type": "line",
    "bars": 252,
    "last_value": 0,
    "first_value": 0,
    "change": 0,
    "change_pct": 0,
    "rendered_at": 0
  },
  "tier_info": {}
}
```

Errors

code	When
NO_DATA	Missing data
NO_CLOSE_DATA	Empty close array
RENDER_ERROR	matplotlib / renderer exception

Note: `script` is accepted for API symmetry but **not executed** on this path today — the chart is pure data visualization.

POST /preview/indicator

Body

```
{
  "expression": "ta.sma(close, 14)",
  "data": { "close": [/* ... */] },
  "options": { "color": "#FF9800", "width": 600, "height": 300 }
}
```

Supported expression prefixes in `_compute_indicator`:



Expression	Series
<code>ta.sma(..., period)</code>	SMA of close
<code>ta.ema(..., period)</code>	EMA
<code>ta.rsi(..., period)</code>	RSI
<code>ta.macd(...)</code>	MACD histogram (12/26/9)
other	falls back to raw <code>close</code>

Parse failures in the helper fall back to plotting `close`.

Internals

Path	Role
<code>backend/api/preview.py</code>	Route handlers + indicator math
<code>backend/services/chart_renderer.py</code>	PNG encode
<code>backend/middleware/auth.py</code>	<code>track_usage</code>

Schemas `PREVIEW_*` in `schemas.py` document intended validation; current handlers use `request.get_json()` or `{} directly.`

Invariants and edge cases

1. **Data shape differs from `/run`**: preview uses **columnar dicts**; `/run` uses **list of bar dicts**.
2. **Not a full Pine interpreter** for indicator expressions — string prefix parsing only.
3. **Width/height clamps** protect worker memory/CPU.
4. Successful responses include `tier_info` from the authenticated key.

Worked example

```
curl -s http://127.0.0.1:5002/preview/chart \
-H "Authorization: Bearer $API_KEY" \
-H 'Content-Type: application/json' \
-d '{
  "data": {"close": [1,2,3,4,5], "volume": [10,10,10,10,10]},
  "options": {"type": "line", "show_volume": true}
}' | jq '.meta'
```

Failure modes

Symptom	Cause
401	Missing key
429	Tier limit
Empty-looking PNG	Renderer empty-chart path when all values null



See also

- [Chart renderer](#)
- [Auth and keys](#)
- [Run](#)



BACKTEST ENDPOINT

POST /backtest/quick — usage-tracked strategy metrics, equity curve, and optional mock OHLCV.

BACKTEST ENDPOINT

Abstract

POST /backtest/quick is a Pro, usage-tracked route that runs a **simplified** strategy simulation over columnar OHLCV (or generated mock bars), returns trade lists + summary metrics, and embeds a base64 equity-curve PNG. It is optimized for speed/demo UX; it is not a full fidelity substitute for event-aware Runtime strategy evaluation on /run.

Conceptual model

```
flowchart TD
    REQ[POST /backtest/quick] --> AUTH[AUTH[track_usage]]
    AUTH --> DATA{data or mock_data?}
    DATA -->|mock / empty close| MOCK[generate_mock_ohlc]
    DATA -->|provided| OHLCV[columnar OHLCV]
    MOCK --> SIM[run_quick_backtest]
    OHLCV --> SIM
    SIM --> METRICS[equity, trades, summary]
    SIM --> PNG[render_equity_curve]
```

Blueprint: backtest_bp, prefix /backtest.

Interface surface

Request

```
{
  "script": "@version=5\nstrategy(\"s\")...",
  "data": {
    "open": [], "high": [], "low": [], "close": [], "volume": []
  },
  "initial_capital": 10000.0,
  "mock_data": false,
  "mock_bars": 252
}
```



Field	Default	Notes
script	required	Empty → NO_SCRIPT
data	{}	Columnar; may be omitted if mock_data
initial_capital	10000	Starting equity
mock_data	false	Force synthetic bars
mockBars	252	Length of mock series

If neither usable close nor mock_data → NO_DATA .

Success

```
{
  "status": "success",
  "result": {
    "equity_curve": [10000, ...],
    "trades": [
      {
        "entry_time": 0,
        "entry_price": 0,
        "exit_time": 0,
        "exit_price": 0,
        "direction": "long",
        "pnl": 0,
        "pnl_pct": 0,
        "size": 1
      }
    ],
    "summary": {
      "total_pnl": 0,
      "total_pnl_pct": 0,
      "sharpe_ratio": 0,
      "max_drawdown": 0,
      "max_drawdown_pct": 0,
      "win_rate": 0,
      "profit_factor": 0,
      "total_trades": 0,
      "winning_trades": 0,
      "losing_trades": 0,
      "avg_win": 0,
      "avg_loss": 0
    },
    "equity_chart": "<base64 png>"
  },
  "tier_info": {},
  "meta": {
    "bars": 252,
    "initial_capital": 10000,
    "completed_at": 0
  }
}
```



Errors

code	HTTP	When
<code>NO_SCRIPT</code>	400	Empty script
<code>NO_DATA</code>	400	No data and not mock
<code>BACKTEST_ERROR</code>	500	Exception in simulation
<code>UNAUTHORIZED</code> / <code>RATE_LIMITED</code>	401 / 429	Auth

Internals

`run_quick_backtest` → `run_backtest` in `backend/services/backtest.py`.

MVP simulation characteristics:

1. Optionally `parse(script)` (errors soft-ignored for MVP path).
2. Precompute long/short entry signals from dual SMA cross (10 vs 20) starting at bar 20.
3. Exit heuristics via RSI-like average and opposite signals / end of series.
4. Equity curve + trade PnL with optional commission/slippage parameters on the lower-level API.
5. Metrics: Sharpe ($\sqrt{252}$ scaling), max drawdown, win rate, profit factor.
6. Chart via `render_equity_curve`.

`generate_mock_ohlc(n_bars)` produces synthetic columns for demos.

Path	Role
<code>backend/api/preview.py</code>	<code>quick_backtest</code> route
<code>backend/services/backtest.py</code>	Simulation + metrics
<code>backend/services/chart_renderer.py</code>	Equity PNG

Invariants and edge cases

1. **Script content is lightly used** in the MVP sim — do not treat results as broker-accurate for arbitrary strategies.
Prefer `/run` events for engine-faithful strategy traces.
2. **Columnar data**, same family as preview, not `/run` bar lists.
3. **Mock path** ignores incomplete user data when `mock_data` or empty close.
4. One usage increment per successful HTTP response under `track_usage`.

Worked example

```
curl -s http://127.0.0.1:5002/backtest/quick \  
-H "Authorization: Bearer $API_KEY" \  
-H 'Content-Type: application/json' \  
-d '{  
  "script": "//@version=5\nstrategy(\"demo\")\n// body optional for MVP",  
  "mock_data": true,  
  "mock_bars": 120,  
  "initial_capital": 25000  
}' | jq '.result.summary'
```



Failure modes

Symptom	Cause
Implausible trades vs script logic	MVP signal model, not full evaluator
Empty trade list	No crosses in series / short history
Missing <code>equity_chart</code>	Renderer exception swallowed → empty string

See also

- [Runtime bridge](#) — faithful strategy events via `/run`
- [Chart renderer](#)
- [Strategy builtins](#)



CHART RENDERER

Matplotlib Agg PNG services: line charts, OHLCV candles, equity curves, base64 encoding.

CHART RENDERER

Abstract

`backend/services/chart_renderer.py` is a headless plotting façade. It forces `matplotlib` into **Agg**, draws dark-theme charts for desk thumbnails, and returns **base64-encoded PNG** strings suitable for JSON transport. Callers are the preview and backtest routes; the module has no Flask dependency.

Conceptual model

```
flowchart LR
    VAL[series / OHLCV / equity] --> PLT[matplotlib Figure Agg]
    PLT --> BUF[BytesIO PNG]
    BUF --> B64[base64 utf-8 string]
    B64 --> API[JSON chart / equity_chart fields]
```

Interface surface

`render_line_chart`

Param	Default	Role
<code>values</code>	required	Y series; <code>None</code> → NaN gaps
<code>dates</code>	<code>None</code>	Optional x tick labels (≤ ~6 ticks)
<code>title</code>	<code>"Chart"</code>	Title string
<code>color</code>	<code>#2196F3</code>	Line + fill
<code>height</code> / <code>width</code>	300 / 600	Pixel intent → <code>figsize</code> <code>/100</code> , dpi 100
<code>show_volume</code>	false	Twin axis bars from <code>ohlcv["volume"]</code>
<code>ohlcv</code>	<code>None</code>	Volume source when enabled

Empty / all-null values → `_render_empty_chart` placeholder.

`render_ohlcv_chart`

Candlestick + volume subplot pair from dict keys `open` , `high` , `low` , `close` , `volume` . Green/red body colors (`#4CAF50` / `#F44336`). Shared x-axis.

`render_equity_curve`

Single series with profit/loss fill relative to zero baseline and a dashed reference at the first equity point. Title fixed `"Equity Curve"` .



Return type

All public renderers → `str` (base64 PNG without data-URI prefix). Clients typically use:

```
data:image/png;base64,{chart}
```

Internals

Detail	Implementation
Backend	<code>matplotlib.use("Agg")</code> before pyplot import
Theme	fig <code>#1E1E1E</code> , axes <code>#252526</code> , muted grids
Cleanup	<code>plt.close(fig)</code> after <code>savefig</code>
Empty state	Centered “No data available” text

Path	Role
<code>backend/services/chart_renderer.py</code>	Renderers
<code>backend/api/preview.py</code>	Consumers
<code>backend/services/backtest.py</code>	Equity chart hook

Invariants and edge cases

1. **Not interactive** — no GUI backend; safe for servers.
2. **NaNs** break lines at gaps (numpy nan).
3. **Candle body height** uses `+ 1e-9` to keep zero-range bodies visible.
4. **Thread safety**: matplotlib global state is historically process-local; multi-threaded gunicorn workers should avoid concurrent pyplot use on one worker or serialize renders.
5. Exceptions in callers often become HTTP `RENDER_ERROR` or empty equity chart strings.

Worked example

```
from backend.services.chart_renderer import render_line_chart

b64 = render_line_chart([1.0, 1.2, 1.1, 1.4], title="demo", color="#A3E635")
assert isinstance(b64, str) and len(b64) > 100
```

Failure modes

Symptom	Cause
ImportError Agg	Incomplete matplotlib install
Huge base64	High width/height — routes clamp preview sizes
Blank image	Empty input path succeeded with placeholder



See also

- [Preview endpoints](#)
- [Backtest](#)

RUNTIME BRIDGE

backend.runtime.Runtime — bar-loop evaluate façade, PineSeries context, compile mode, and result packaging.

RUNTIME BRIDGE

Abstract

`backend.runtime.Runtime` is the HTTP stack's **evaluate façade**. It owns symbol metadata namespaces (`syminfo`, `timeframe`, `barstate`, ...), walks OHLCV bars, updates `PineSeries` for OHLC, visits a shared `CustomEvaluator` on the parsed AST each bar, drains strategy events, and packages multi-plot series for AXIS. Compile mode optionally swaps the AST walker for a Numba-backed subset engine.

This is the bridge between Flask handlers and the language core — not a second semantics.

Conceptual model

```

flowchart TB
    SRC[Pine source] --> PARSE[parse mode=exec]
    OHLCV[bar list] --> LOOP[for each bar]
    PARSE --> LOOP
    LOOP --> SER[PineSeries.update OHLC]
    LOOP --> CTX[context bar_index time barstate]
    LOOP --> FILL[process_pending_orders]
    LOOP --> VISIT[evaluator.visit tree]
    VISIT --> PLOTS[plot_outputs]
    VISIT --> EV[strategy events]
    LOOP --> PACK[series map + plot_meta + drawings]
```

Interface surface

Construction

```
Runtime(symbol: str = "AAPL", run_id: str | None = None)
```

- Sets `syminfo.tickerid` / `name` / `prefix` (prefix from `EXCHANGE:SYMBOL` form).
- Generates `run_id` as `uuid4().hex[:16]` when omitted.

Helpers: `configure_footprint`, `update_bid_ask`.



```
run(source_code, ohlcv_data, data_feed=None, data_provider=None, mode="interpret")
```

Arg	Role
source_code	Pine text
ohlcv_data	list[dict] with open/high/low/close/time[/volume/bid/ask]
data_feed / data_provider	request.* wiring; auto-resolved from chart bars when unset
mode	"interpret" (default) or "compile"

Interpret path (default)

1. Resolve request sources (non-fatal on failure).
2. `parse(source_code, mode="exec")` — on failure `{ "error": "Parse Error: ..." }`.
3. Init `PineSeries` for OHLC + context dict (timeframe daily defaults, barstate, chart colors).
4. Construct `CustomEvaluator(context=..., data_feed=..., data_provider=...)`, reset var declarations + drawing registry.
5. Per bar:
 - Update series + calendar fields from timestamp
 - Update barstate flags (`isfirst` / `islast` / history confirmed)
 - `process_pending_orders` when available
 - `evaluator.visit(tree)` — on failure `{ "error": "Runtime Error at bar ..." }`
 - Lock pine defs after first bar (`_pine_defs_locked`) to avoid multi-dispatch blow-ups
 - Drain strategy events; stamp `script_id` / `run_id`
 - Capture plot outputs
6. Build `series` / `plot_meta` from all plot indices (disambiguate duplicate titles with `_2` suffixes).
7. Export drawings via `DrawingRegistry.export_for_api(bar_times)`.

Return keys: `plots`, `series`, `plot_meta`, `events`, `drawings`, `count`, `script_id`, `run_id`, `mode`.

Compile path

1. Import `pynescrypt.compiler.engine` — missing → error string.
2. Require `has_numba()`.
3. `compile_script` + `compiled.run(opens, highs, lows, closes, volumes)`.
4. Pop internal keys (`__drawings`, `__events`, position meta).
5. Return similar envelope with `mode: "compile"`, plus `object_mode` / `generated_code` when present.

PineSeries (backend/series.py)

History deque (default maxlen 1000), `series[0]` current / `series[n]` lookback, arithmetic on **current** values, `None` propagates as na-like absence. Negative indices raise — Pine does not allow them.

Context namespaces

Defined in `runtime.py`: `Syminfo`, `Chartinfo`, `Timeframe`, `Barstate`, `Chart` — attribute names aim at Pine v5/v6 surface (`timeframe.isdaily`, `syminfo.isin`, ...).



Internals

Path	Role
backend/runtime.py	Runtime
backend/evaluator.py	CustomEvaluator specialization
backend/series.py	PineSeries
src/pynescrypt/ast/helper.py	parse
src/pynescrypt/ast/evaluator/**	Builtin + strategy semantics
src/pynescrypt/compiler/engine.py	Compile mode

Flask wiring: `backend/app.py` constructs a **fresh** `Runtime` per request (and per batch job) so `run_id` and drawing registries do not leak across users.

Invariants and edge cases

1. **Parse once, visit many** — AST is not re-parsed per bar.
2. **Defs locked after bar 0** — performance invariant for large function tables.
3. **Drawing registry reset** at run start — isolation between requests.
4. **Primary plots list** is plot index 0 for backward compatibility; AXIS should prefer `series` + `plot_meta`.
5. **Errors are dicts**, not exceptions, for control-flow at the HTTP boundary (`"error" in result`).
6. Compile mode supports a **subset** (documented in engine — `ta/math/plots` family); unsupported scripts should use interpret.

Worked example

```
from backend.runtime import Runtime

rt = Runtime(symbol="NASDAQ:AAPL")
out = rt.run(
    '//@version=5\nindicator("x")\nplot(close, "C")',
    [{"time": i, "open": 1, "high": 2, "low": 0.5, "close": 1.0 + i * 0.01, "volume": 1} for i in range(50)]
)
assert "error" not in out
assert out["count"] == 50
assert "C" in out["series"] or "plot_0" in out["series"]
```

Failure modes

Message prefix	Cause
Parse Error:	Grammar / syntax
Runtime Error at bar	Evaluator exception
Order fill error at bar	Broker sim pending orders
Compile mode requires numba	Missing dependency
Compile Error: / Compiled Runtime Error:	Engine path



See also

- [Contract](#)
- [Run endpoint](#)
- [Series and history](#)
- [Strategy events](#)



EVALUATE CONTRACT

Shared request/response evaluate contract across Flask Pro API, pyne-worker, and pine-worker.

EVALUATE CONTRACT

Abstract

The **evaluate contract** is the JSON dialect that lets AXIS, HOOX, CLI tools, and edge workers call *any* PYNE-compatible host interchangeably. Flask `POST /run` is the reference implementation; Python **pyne-worker** and TypeScript **pine-worker** aim for the same fields even when transport (Worker `fetch`, queue, etc.) differs.

This page freezes the semantic envelope — not transport auth headers.

Conceptual model

```
flowchart LR
    subgraph Producers
        AXIS[AXIS]
        CLI[CLI / scripts]
        HOOX[HOOX mesh]
    end
    subgraph Hosts
        FLASK[Flask /run]
        PYW[pyne-worker]
        TSW[pine-worker]
    end
    AXIS --> FLASK
    AXIS --> PYW
    AXIS --> TSW
    CLI --> FLASK
    HOOX --> PYW
    HOOX --> TSW
```

Invariant: given the same script + OHLCV + mode, hosts should agree on plot series and strategy events within known parity limits (see [numerical validation](#) and pine-worker testing).



Interface surface

Request (single evaluate)

Field	Type	Required	Description
script	string	yes	Full Pine source
data	Bar[]	yes	Chronological OHLCV
symbol	string	no	Default CHART / host-specific
mode	"interpret" "compile"	no	Default interpret
data_source	string	no	Optional external history provider id
data_options	object	no	Provider configuration

Bar

Field	Type	Required	Notes
time	number	recommended	Unix ms preferred
open	number	yes	
high	number	yes	
low	number	yes	
close	number	yes	
volume	number	no	Default host-dependent
bid / ask	number	no	Runtime bid/ask hooks

Response (success)

Field	Type	Description
status	"success"	Flask sets this; workers may omit and use HTTP 200 only — prefer including it
plots	(number null)[]	Primary series (plot 0)
series	Record<string, (number null)[]>	Named multi-series
plot_meta	Record<string, PlotMeta>	Display metadata
events	StrategyEvent[]	Strategy / trade-like events
drawings	Drawing[]	line/label/box export
count	number	Bars evaluated
script_id	string	Stable hash prefix of source
run_id	string	Per-invocation id
mode	string	Echo effective mode
data_source	string	Flask echo of resolved source label



PlotMeta (Flask)

```
{ "title": "C", "color": "#2196F3", "linewidth": 1, "index": 0 }
```

StrategyEvent (minimum)

Hosts should preserve:

```
{
  "type": "string",
  "bar_index": 0,
  "script_id": "...",
  "run_id": "..."
}
```

Additional fields follow evaluator `to_dict()` / TS port parity — treat unknown keys as forward-compatible.

Response (failure)

Flask `/run`:

```
{
  "status": "error",
  "code": "EXECUTION_ERROR",
  "message": "Runtime Error at bar ..."
}
```

Workers may use equivalent `{ error: string }` or HTTP 4xx/5xx with message body. Clients should accept:

1. Envelope `status === "error"` with `message`, or
2. Presence of top-level `error` string (raw `Runtime.run` shape).

Batch evaluate (Flask extension)

POST `/run/batch`:

Request: `scripts: (string | {id, script})[]` (max 8) + shared `data` + same optional fields as single run.

Response:

```
{
  "status": "success" | "partial",
  "results": [ /* per-script success or error object with id */ ],
  "count": 0,
  "ok": 0,
  "data_source": "chart"
}
```

Workers may implement batch as N single evaluates; AXIS should not assume batch exists on every host.



Internals — reference mapping

Contract field	Flask source
plots / series / plot_meta	Runtime.run packing loop
events	evaluator._strategy_state.drain_events() → to_dict()
drawings	DrawingRegistry.export_for_api
script_id	sha256(source)[:16]
run_id	Runtime._run_id

Schema enforcement on Flask free tier: `RUN_SCHEMA` / `RUN_BATCH_SCHEMA` in `backend/middleware/schemas.py` (strict, reject extras).

Tests: `tests/test_backend.py` (`test_run_success` , batch cases).

Divergences to remember

Surface	Divergence
Preview / backtest	Columnar <code>data</code> dict, not <code>Bar[]</code> ; not the evaluate contract
Compile mode	Extra keys <code>object_mode</code> , <code>generated_code</code> may appear
Auth	Flask free <code>/run</code> open; workers may require HOOX mTLS / API gateways
Error codes	Flask uses <code>code</code> enums; workers should map to stable strings when possible

Invariants and edge cases

1. **Bar order is chronological ascending** — hosts do not sort for you.
2. `na` / `missing` appear as JSON `null` in series arrays.
3. **Duplicate plot titles** get suffixes (`_2` , ...) on Flask; clients should key on returned map keys, not assumed titles.
4. **Idempotent script_id** for identical source text across hosts.
5. **run_id is not stable** across retries — use for log correlation only.
6. Parity is best-effort on floating edges; pin fixtures when testing hosts against each other.

Worked example — host-agnostic client sketch

```
type EvaluateRequest = {
  script: string;
  data: Array<{
    time: number;
    open: number;
    high: number;
    low: number;
    close: number;
    volume?: number;
  }>;
  symbol?: string;
  mode?: "interpret" | "compile";
};

type EvaluateSuccess = {
  plots: Array<number | null>;
  series: Record<string, Array<number | null>>;
  events: Array<Record<string, unknown>>;
  drawings: Array<Record<string, unknown>>;
  count: number;
  script_id: string;
  run_id: string;
  mode: string;
};
```

POST that body to Flask `/run` or the worker’s evaluate route; branch on `status` / `error`.

Failure modes

Client bug	Symptom
Sending columnar preview data to <code>/run</code>	Schema <code>data</code> must be a list
Assuming batch on worker	404 / unknown field
Ignoring <code>series</code> and only reading <code>plots</code>	Multi-plot scripts look single-series

See also

- [Run endpoint](#)
- [Runtime bridge](#)
- [pine-worker](#)
- [Pro API usage \(end user\)](#)
- [Events](#)